Operating Systems
Sheet 4 Solutions

Name: Yehia Said Gewily
ID: 21010705
Date: March 2026

1 Processes and Threads (4.1)

1. Process vs. Thread

Overhead in Creation & Deletion:

- **Process:** Creating a process is expensive. The OS must allocate a brand new address space, copy (or set up copy-on-write) memory pages, duplicate file descriptors, create a PCB, and initialize kernel data structures. Deletion is equally heavy. Think of it as building a whole new apartment from scratch.
- **Thread:** A thread lives *inside* an existing process and shares its memory space. Creating one only requires a new stack and a small thread-control block (TCB). This makes thread creation and deletion much cheaper and faster — typically 10–100× faster than process creation.

Communication and Speed:

- **Process Communication (IPC):** Processes have completely separate memory spaces, so to share data they need OS-provided mechanisms such as pipes, message queues, shared memory segments, or sockets. These all involve system calls and kernel intervention, making them relatively slow.
- **Thread Communication:** Threads within the same process share the same heap, global variables, and open files directly. They can communicate just by reading and writing shared memory — no kernel calls needed. This is extremely fast. The trade-off is the need for synchronization (mutexes, semaphores) to prevent race conditions.

2 Types of Threads (4.2)

2. User-Level vs. Kernel-Level Threads

Aspect	User-Level Threads (ULT)	Kernel-Level Threads (KLT)
Mapping	Many-to-one: many user threads map to a single kernel thread. The kernel only sees the process.	One-to-one: each user thread has a corresponding kernel thread. The OS is aware of every thread.
Multi-processor	Cannot run in parallel. Since only one kernel thread exists, the OS always runs it on one core at a time regardless of how many CPUs are available.	Can achieve true parallelism. Different threads can be scheduled on different cores simultaneously.
Kernel Overhead	Very low. Thread creation, switching, and synchronization happen entirely in user space — no system calls needed.	Higher. Every thread operation (create, switch, sync) requires a mode switch into the kernel.
Portability	Highly portable. Implemented as a user-space library, so it works on any OS regardless of kernel support.	Less portable. Directly depends on the kernel's thread API, which varies between operating systems.
Scheduling	Done by the thread library (user-space scheduler). The kernel scheduler is completely unaware of individual threads.	Done by the kernel scheduler. The OS manages and schedules each thread directly, just like a process.

3. Why Do All Threads Block When a ULT Makes a System Call?

With user-level threads, the kernel has no idea that individual threads exist — it only sees the process as a single schedulable entity. So when one user thread makes a blocking system call (like reading from disk), the kernel blocks the *entire process*, since that is the only unit it knows about. Because all threads share that process, they all get blocked together even though they had nothing to do with the system call.

Does this happen with kernel-level threads?

No. With KLTs, each thread is independently known to the kernel. If one thread blocks on a system call, the kernel simply switches to another ready thread within the same process. The remaining threads continue executing normally. This is one of the main advantages of kernel-level threads.

4. Advantages and Disadvantages: ULT vs. KLT

User-Level Threads — Advantages:

- Much faster to create, switch, and destroy — no kernel system calls needed.
- Fully portable: implemented as a library that runs on any operating system.
- Thread switching does not require a user-to-kernel mode switch, saving CPU time.
- The application can implement its own custom scheduling policy.

User-Level Threads — Disadvantages:

- One blocking system call blocks *all* threads in the process.
- Cannot exploit multiple processors — the OS sees only one unit of execution.
- A page fault by one thread blocks the entire process.

Kernel-Level Threads — Advantages:

- True parallelism: different threads run on different CPUs at the same time.
- Blocking one thread does not affect others in the same process.
- The OS can schedule threads intelligently across all available processors.

Kernel-Level Threads — Disadvantages:

- Higher overhead: creating and switching threads requires system calls.
- Less portable: tightly coupled to OS-specific kernel APIs.
- Thread management is generally slower compared to user-level threads.

3 Multicore and Multithreading (4.3)

5. Thread Switch vs. Process Switch — Which is Faster?

Thread switch is faster, and by a significant margin. Here is why:

- **Thread switch (same process):** Threads share the same address space. Switching between them only requires saving/restoring the thread's registers and stack pointer. The memory map (page tables) stays identical, so the TLB does *not* need to be flushed. File handles and resources are shared — nothing changes there.
- **Process switch:** Requires saving the entire CPU context, swapping out the whole memory map to load a completely different page table, *flushing the TLB* (which is expensive — all cached translations are lost), and loading the new process's PCB. The cold TLB then causes many expensive page-table lookups until it warms up again.

In short: a thread switch changes the execution point; a process switch changes the entire execution environment.

6. Threaded vs. Non-Threaded for Independent Concurrent Calculations

Take matrix multiplication as an example: rows can be computed completely independently, making it a good candidate for parallelism. Whether threading actually helps depends on the thread type and hardware.

User-Level Threads:

- **Uniprocessor:** No speedup. ULTs are time-shared on a single CPU. The parallelism is only apparent. For CPU-bound work like matrix math, the thread-switching overhead actually makes it slightly *slower* than a simple single-threaded solution.
- **Multiprocessor:** Still no speedup. Since all ULTs map to a single kernel thread, the OS always runs them on one core. The extra processors go unused.

Kernel-Level Threads:

- **Uniprocessor:** Marginal benefit at best. Only one thread runs at a time, so there is no true parallelism. The added overhead of kernel-managed thread switches slightly hurts performance for CPU-bound tasks.
- **Multiprocessor: Significant speedup.** Each kernel thread can run on a separate core simultaneously. Independent row computations of the matrix can be distributed across all available CPUs, giving near-linear speedup. This is the ideal scenario for kernel-level multithreading.

Conclusion: Threading genuinely helps only when using kernel-level threads on a multiprocessor system. For CPU-bound tasks on a uniprocessor, threading adds overhead without benefit. User-level threads never help for this type of workload.

4 Windows Process and Thread Management (4.4)

7. Thread States in the Windows OS

Windows defines seven thread states:

- **Initialized:** An internal state used only during thread creation, before the thread is placed in any queue. Not visible to the scheduler.
- **Ready:** The thread is fully prepared to run and is waiting to be assigned to a processor. All runnable threads that are not currently executing compete in this state.
- **Standby:** One specific thread has been *selected* to run next on a particular processor. Only one thread per processor can be in Standby at a time. It transitions to Running when the processor becomes free.
- **Running:** The thread is currently executing on a processor. It continues until it is preempted by a higher-priority thread, its time quantum expires, it blocks, or it terminates.
- **Waiting:** The thread is blocked — it issued a blocking operation (I/O, sleep, waiting on a mutex/event). It cannot run until the waited-upon event is signaled.
- **Transition:** The thread is logically ready to run, but its kernel stack has been paged out to disk to free memory. Once the stack is paged back in, the thread moves to Ready.
- **Terminated:** The thread has finished execution. Its object may be kept alive if another thread holds a handle to it; otherwise its resources are reclaimed.

5 General Questions

8. Tape Drive Allocation (8 Processors, 20 Tape Drives)

Setup: 8 processors, 20 tape drives. Each job needs at most 4 drives: it uses 3 for most of its execution, then needs a 4th briefly at the end.

Part a — Policy: Hold 4 drives per job from start to finish.

- **Maximum jobs in progress:** $\lfloor 20 \div 4 \rfloor = \mathbf{5}$ jobs.
- **Maximum idle drives:** In the worst case, all 5 running jobs are in their early phase and each is only using 3 of its 4 allocated drives. That leaves $5 \times 1 = \mathbf{5}$ idle drives.
- **Minimum idle drives:** If all 5 jobs happen to be in their final phase and using all 4 drives: $5 \times 4 = 20$ drives in use $\Rightarrow$ **0** idle drives.

This policy is wasteful — drives sit idle most of the time while jobs are in their first phase.

Part b — Better Policy: Allocate drives on demand.
Assign only 3 drives when a job starts. Grant the 4th drive only when the job actually requests it. Release all drives when the job finishes.

- **Deadlock avoidance:** To prevent deadlock (every job holding 3 drives and each waiting for a 4th that will never be freed), we must guarantee at least one job can always get its 4th drive and finish. With 20 drives, we can allow at most 6 jobs to start (using $6 \times 3 = 18$ drives), keeping 2 drives spare. Those 2 spare drives are enough to give one job its 4th drive, let it finish, and release its 4 drives for the others. This breaks the deadlock cycle.
- **Maximum jobs in progress:** $\lfloor 20 \div 3 \rfloor = \mathbf{6}$ jobs (with 2 drives spare).
- **Bounds on idle drives:** Minimum $= 0$ (when multiple jobs are in their final phase using all 4 drives). Maximum ≈ 2 drives idle (when all 6 jobs are in early phase: $6 \times 3 = 18$ drives used, 2 spare). Drive utilization is significantly better than policy (a).

9. Many-to-Many Threading on a Multiprocessor

Setup: Multiprocessor system. Program uses many-to-many threading (user threads > processors). We analyse three scenarios based on the number of kernel threads allocated.

a. Kernel threads < number of processors:

Some processors will be left idle because there are not enough kernel threads to keep them all busy. The program is under-utilizing the available hardware. Even if there are many user threads ready to run, they cannot all execute in parallel because there are not enough kernel-thread "slots" to map them to. **Performance is limited by the number of kernel threads, not processors.**

b. Kernel threads = number of processors:

This is the **optimal scenario**. Every processor has exactly one kernel thread to run.

All CPUs are kept busy (assuming enough ready user threads exist), and no processor sits idle. There is no unnecessary contention or context switching between kernel threads on the same core. **Maximum hardware utilization with minimum overhead.**

c. Kernel threads > processors but < user threads:

Now there are more kernel threads than processors. Multiple kernel threads compete for the same core, causing unnecessary kernel-level context switching. This introduces overhead without additional parallelism — you can still only run as many threads simultaneously as there are processors. However, since there are still fewer kernel threads than user threads, not all user threads can be concurrently mapped. **Performance is slightly degraded compared to (b) due to extra scheduling overhead.**

10. POSIX Threads Program Analysis

Part a — What does the program do?

The program creates one additional thread alongside the main thread. Both threads run concurrently:

- **The child thread** (`thread_function`) loops 20 times: reads `myglobal`, adds 1, prints a dot "`.`", sleeps 1 second, then writes the incremented value back.
- **The main thread** loops 20 times: directly increments `myglobal` by 1 and prints "`o`", sleeping 1 second each iteration.

The intention is that both threads each increment `myglobal` 20 times, for a final value of 40. The main thread then waits for the child with `pthread_join` and prints the result.

Part b — Is `myglobal equals 21` the expected output?

No, the expected value is 40, not 21. What went wrong is a **race condition** (a data race) on the shared variable `myglobal`.

Here is what happens in the child thread:

1. It reads `myglobal` into a local variable `j`: `j = myglobal;`
2. It increments `j`: `j = j + 1;`
3. It **sleeps for 1 second** before writing back.
4. It writes: `myglobal = j;`

During that 1-second sleep, the main thread can increment `myglobal` many times. But when the child thread wakes up and writes `j` back, it **overwrites all of the main thread's updates** with the stale value it read at step 1.

Example: At some point, `myglobal = 10`. The child reads `j = 10`, increments to `j = 11`, then sleeps. During that sleep, the main thread runs 9 iterations, making `myglobal = 19`. The child wakes and writes `myglobal = 11` — wiping out 8 increments. This lost-update problem repeats throughout execution.

Fix: Protect the read-modify-write sequence with a mutex:

- Declare a `pthread_mutex_t` and lock it around the read and write of `myglobal` in both threads, so only one thread accesses it at a time. The sleep should be *outside* the critical section.